

THE UNIVERSITY OF
MELBOURNE

Final Report

The Art of Scientific Computation Power Laws and Finite Size Scaling in the BTW Abelian Sandpiles Model

Author:
Jonathan Purcell

Subject Handbook code:
COMP90072

Faculty of Science
The University of Melbourne

Draft Version: 9th October 2026

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Mathematical Formulation of the BTW Model . . . 2
1.3 Probabilistic Framework . . . 3
1.4 Statistical Models . . . 4

2 Simulation **5**
2.1 Method . . . 5
2.2 Practicalities . . . 5
2.3 Results . . . 6

3 Power Law Fit **8**
3.1 Bounded Power Law Fit . . . 8
3.2 Manually-truncated Power Law . . . 11

4 Finite Size Scaling Effect **13**
4.1 Exponentially Truncated Power Law . . . 13
4.2 Pooled Exponentially Truncated Power Law . . . 16
4.3 Stretched Exponentially Truncated Power Law . . . 16

5 Conclusion **22**
5.1 Reflection . . . 22

Appendix A Extended MLE Fit Plots **23**

Bibliography **23**

1

Introduction

1.1 Motivation

1. Introduce the BTW sandpiles model as a model for self-organised criticality and 1/f noise [1]
2. Define criticality, self-organised criticality, power law scaling, etc

1.2 Mathematical Formulation of the BTW Model

The Abelian Sandpiles Model (or BTW Sandpile Model) was introduced by Bak, Tang, and Wiesenfeld in 1987 [2] as an example of a dynamical system driving itself towards criticality, exhibiting what they call "self-organised criticality".

We introduce the sandpile model, adapting the notation of Járai, 2018 [5].

Define a two-dimensional square lattice with lattice size L as the a graph $G_L = (V_L, E_L)$, where

$$V_L := \{(x, y) \in \mathbb{Z}^2 : 1 \leq x, y \leq L\}$$

defines the set of vertices and

$$E_L := \{\mathbf{xy} : \mathbf{x}, \mathbf{y} \in V_L \wedge \|\mathbf{x} - \mathbf{y}\|_1 = 1\}$$

defines the set of edges, where $\|\mathbf{x} - \mathbf{y}\|_1$ denotes the L_1 norm, i.e. $\|(x_1, x_2) - (y_1, y_2)\|_1 = |x_1 - y_1| + |x_2 - y_2|$.

Let $N(\mathbf{x}) = \{\mathbf{y} \in V_L : \mathbf{xy} \in E_L\}$ be the neighbourhood of $\mathbf{x}$ in G.

A sandpile is a map $\eta : V_L \to \mathbb{Z}_{\geq 0}$, and $\eta(\mathbf{x})$ is called the height at site $\mathbf{x}$.

Define the critical height as $\eta_c := \max_{\mathbf{x} \in V_L} \deg_G(\mathbf{x})$. In the above setup, $\eta_c = 4$ provided $L \geq 3$. A site $\mathbf{x} \in V_L$ is called stable iff $\eta(\mathbf{x}) < \eta_c$ and is otherwise unstable. The sandpile η is stable iff it is stable at each site $\mathbf{x} \in V_L$.

Let $U_\eta = \{\mathbf{x} \in V_L : \eta(\mathbf{x}) \geq \eta_c\}$ be the set of unstable sites.

Let $\Psi_{G_L} = \mathbb{Z}_{\geq 0}^{V_L}$ be the set of possible sandpiles and let $\Omega_{G_L} = \{0, 1, 2, ..., \eta_c - 1\}^{V_L} \subseteq \Psi_{G_L}$ be the set of stable sandpiles.

A toppling is a function $\phi : \Psi_{G_L} \times V_L \to \Psi_{G_L}$ mapping $(\eta, \mathbf{x}) \mapsto \eta'$ where

$$\eta'(\mathbf{y}) = \begin{cases} \eta(\mathbf{y}) + 1 & \mathbf{y} \in N(\mathbf{x}) \wedge \mathbf{x} \in U_\eta \\ \eta(\mathbf{y}) - \eta_c & \mathbf{y} = \mathbf{x} \in U_\eta \\ \eta(\mathbf{y}) & \text{otherwise} \end{cases}$$

Note that we allow open boundary conditions in this study. When a toppling is initiated at a boundary site, we have $\ell(\eta, \mathbf{x}) := \sum_{\mathbf{x} \in V_L} \eta(\mathbf{x}) - \sum_{\mathbf{x} \in V_L} \eta'(\mathbf{x}) > 0$. This is called the loss associated with toppling η at $\mathbf{x}$.

The avalanche (stabilising) function $\xi : \Psi_{G_L} \to \Omega_{G_L}$ maps $\eta = \eta^{(0)} \mapsto \eta^{(s)}$ where $\eta^{(s)}$ is the stable sandpile output by the following algorithm:

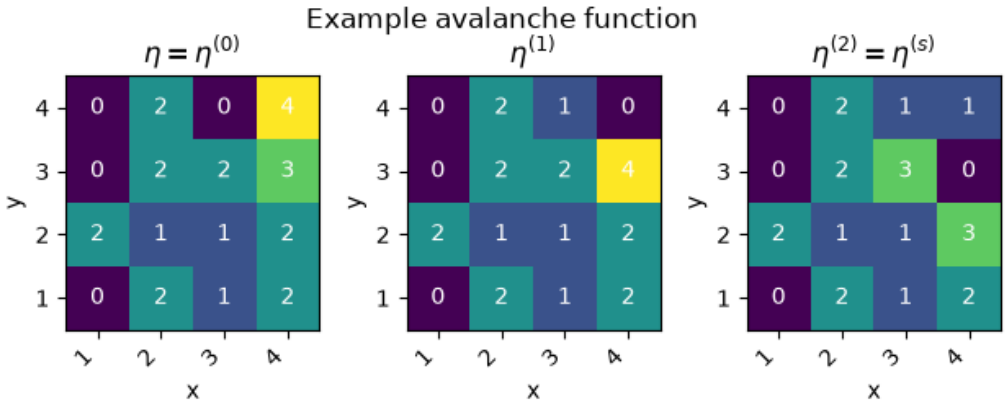


Figure 1.1: Example computation of $\xi(\eta)$ for an unstable sandpile with lattice size $L = 4$ and $\eta_c = 4$. Numbers written at each grid cell represent the sandpile height at vertex (x, y), and correspond to the cell colour. η is first toppled at site $\mathbf{x}^{(1)} = (4, 4)$ to produce $\eta^{(1)}$, which is also unstable. $\eta^{(1)}$ is subsequently toppled at site $\mathbf{x}^{(2)} = (4, 3)$ to produce $\eta^{(2)} \in \Omega_{G_L}$. The algorithm terminates with $s(\eta) = 2$. In the above example, we have $a(\eta) = 2$, $\ell(\eta) = 2 + 1 = 3$, and $d(\eta) = 1$.

Algorithm: Avalanche Function
$i \leftarrow 0$
while $U_{\eta^{(i)}} \neq \emptyset$ **do**:
choose $\mathbf{x}^{(i+1)} \in U_{\eta^{(i)}}$
$\eta^{(i+1)} \leftarrow \phi(\eta^{(i)}, \mathbf{x}^{(i+1)})$
$i \leftarrow i + 1$
end while
return $\eta^{(i)} = \eta^{(s)}$

When $\eta^{(s)} \neq \eta$, we say that an avalanche has occurred.

Avalanches can be described by a range of properties:

1. The avalanche (toppling) size is the final index $s(\eta) = s$
2. The avalanche (toppling) area $a(\eta)$ is the size of the set $\{\mathbf{x}^{(i)} : i \in \{1, \cdots, s\}\}$
3. The loss is $\ell(\eta) := \sum_{i=1}^{s} \ell(\eta^{(i-1)}, \mathbf{x}^{(i)})$
4. The L_1 length is $d(\eta) := \max_{i \in \{1,\cdots,s\}} \left\|\mathbf{x}^{(1)} - \mathbf{x}^{(i)}\right\|_1$.

When an avalanche occurs, note the following constraints:

1. $0 < a(\eta) \leq s(\eta)$
2. $a(\eta) \leq L^2$
3. $0 \leq \ell(\eta) \leq \sum_{\mathbf{x} \in V_L} \eta(\mathbf{x})$
4. $0 \leq d(\eta) \leq 2L$

When no avalanche occurs, we define $s(\eta) = a(\eta) = \ell(\eta) = d(\eta) = 0$ for convenience. However, our focus in this project is on the distribution of these statistics conditional on the occurrence of an avalanche.

A proof that ξ is well-defined and independent of the choice of $\mathbf{x}^{(j)}$ in the loop can be found in Járai, 2018 [5]. The abelian nature of the topplings were first shown by Dhar in 1990 [4].

1.3 Probabilistic Framework

Define the deposition function $\delta : \Psi_{G_L} \times V_L \to \Psi_{G_L}$ as the map $\eta \mapsto \eta'$ with rule

$$\eta'(\mathbf{y}) = \begin{cases} \eta(\mathbf{y}) + 1 & \mathbf{y} = \mathbf{x} \\ \eta(\mathbf{y}) & \mathbf{y} \neq \mathbf{x} \end{cases}$$

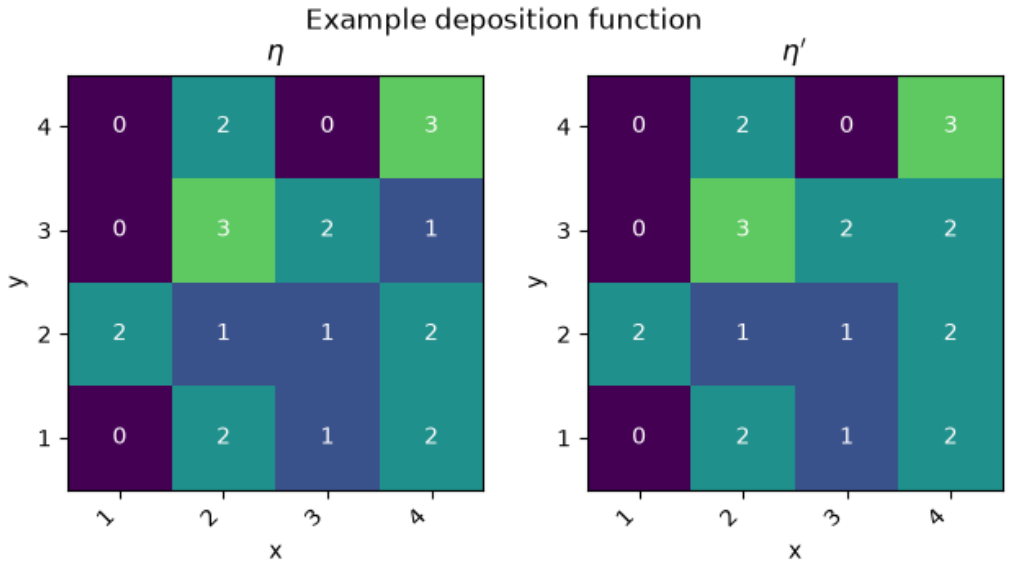


Figure 1.2: Example deposition function δ applied to a sandpile with lattice size $L = 4$. Numbers written at each grid cell represent the sandpile height at vertex (x, y), and correspond to the cell colour. A grain is deposited at site $\mathbf{x} = (4, 3)$, increasing the site height from 1 to 2 and producing η'.

We describe the sandpile evolution process by considering the sequence of iid random variables $\mathbf{X}_1, \mathbf{X}_2 ...$, each with support V_L and pmf

$$p(\mathbf{X}_j = \mathbf{x}) = L^{-2}$$

This represents a sequence of uniformly chosen random vertices, which are taken as locations for grain deposition.

From the sequence $\{X_j\}_{j\geq 1}$, we can build a sequence of stable sandpiles $\{\eta_j\}_{j\geq 0}$ with $\eta_0 = \mathbf{0}$ and $\eta_j = \xi(\delta(\eta_{j-1}, \mathbf{X}_j))$ for $j \geq 1$.

Likewise, we obtain the sequence of avalanche sizes $\{S_j\}_{j\geq 1}$ where $S_j := s(\delta(\eta_{j-1}, \mathbf{X}_j))$. A similar construction yields sequences of avalanche areas, losses, and L_1 lengths.

The sequence $\{\eta_j\}_{j\geq 0}$ forms a discrete Markov Chain with finite state space Ω_{G_L}; the distribution of η_{j+1} depends only on the previous state η_j and the transition probability is determined by the distribution of $\mathbf{X}_j$.

Jarai [5] discusses the Markov structure in detail, and proves that the distribution of states η_j converges to a stationary distribution $\pi(\eta)$ as $j \to \infty$, where $\pi(\eta) > 0$ iff η is a recurrent state in the Markov Chain.

Consequently, the sequence of avalanche sizes $\{S_j\}_{j\geq 1}$ and the other avalanche statistic sequences, which are functions of the states η_j, also converge in distribution to stationary distributions as $j \to \infty$.

1.4 Statistical Models

Describe modelling choices for power law and truncated power law distributions

2

Simulation

2.1 Method

The stationarity of the avalanche statistic sequences forms the theoretical foundation for empirical simulation and analysis according to the following algorithm:

Algorithm: Simulating the Sandpile Model
initialise $\mathbf{s}, \mathbf{a}, \ell, \mathbf{d}$ as empty arrays of size n
initialise $\eta = \mathbf{0}$
for $i \in \{1, \cdots, m\}$ **do**:
simulate $\mathbf{X}_i$
$\eta \leftarrow \delta(\eta, \mathbf{X}_i)$
$\eta \leftarrow \xi(\eta)$
end for
for $i \in \{m+1, \cdots, m+n\}$ **do**:
simulate $\mathbf{X}_i$
$\eta \leftarrow \delta(\eta, \mathbf{X}_i)$
$\mathbf{s}[i-m] \leftarrow s(\eta)$
$\mathbf{a}[i-m] \leftarrow a(\eta)$
$\ell[i-m] \leftarrow \ell(\eta)$
$\mathbf{d}[i-m] \leftarrow d(\eta)$
$\eta \leftarrow \xi(\eta)$
end for
return $\mathbf{s}, \mathbf{a}, \ell, \mathbf{d}$

We can then discard the zero values of the returned arrays, which are not associated with an avalanche.

In the above, simulated states $\eta_1, \cdots, \eta_m$ are called burn-in states, and are discarded. After this period, we assume that the distribution of η_j is approximately stationary. Therefore, simulations $\eta_{m+1}, \cdots, \eta_{m+n}$ are retained and their empirical distribution approximates the stationary distribution $\pi(\eta)$. From that sequence, we construct simulated sequences for each avalanche statistic of length n, whose distribution approximates its respective stationary distribution.

2.2 Practicalities

In practice, hundreds of thousands of simulations are required to approach stationarity. This is computationally expensive, especially for large lattice sizes, because computations cannot be parallelised due to the Markov structure of the problem. An initial Python implementation of the simulation procedure above was too slow to produce meaningful results. The final implementation uses `numba` with `njit` decorators, which stands for 'no python just in time' [6]. This compiles the Python code prior to execution, bypassing the CPython interpreter and achieving execution speed near code written in C. However, only a subset of Python and `numpy` functionality is permitted, requiring significant refactoring away from object-oriented design. Table 2.1 compares identical implementations of the simulation for varying lattice sizes, where one uses `numba @njit` decorators, while the other does not. Results show a 100-140x speedup using the optimised code.

L	`numba`-optimised runtime (s)	standard runtime (s)	speedup
5	0.03	3.90	135.15
7	0.05	5.93	117.16
10	0.09	9.41	104.69
12	0.12	11.68	99.98
15	0.13	17.62	132.55

Table 2.1: Runtime comparison between the `numba`-based implementation and the standard Python implementation. The optimised code runs 100-140x faster. Simulations comprised 100,000 depositions (40,000 burn-in and 60,000 subsequent) and were performed on lattice sizes $L \in \{5, 7, 10, 12, 15\}$.

2.3 Results

Simulations were performed for lattice sizes $L \in \{5, 10, 15, 20, 30, 40, 50, 65, 80, 100\}$. The burn-in period was set to $m = 4 \times 10^4$ and sequences of length $n = 1 \times 10^6$ were retained for each lattice size.

The plots in Figure 2.1 depect a linear trend in early values of avalanche size, area, and L_1 distance, suggesting a power law distribution followed by a sharp decay. The distribution for Loss does not appear to have this power law structure, and the remainder of this report focusses on understanding and analysing the distribution of avalanche size, area, and length.

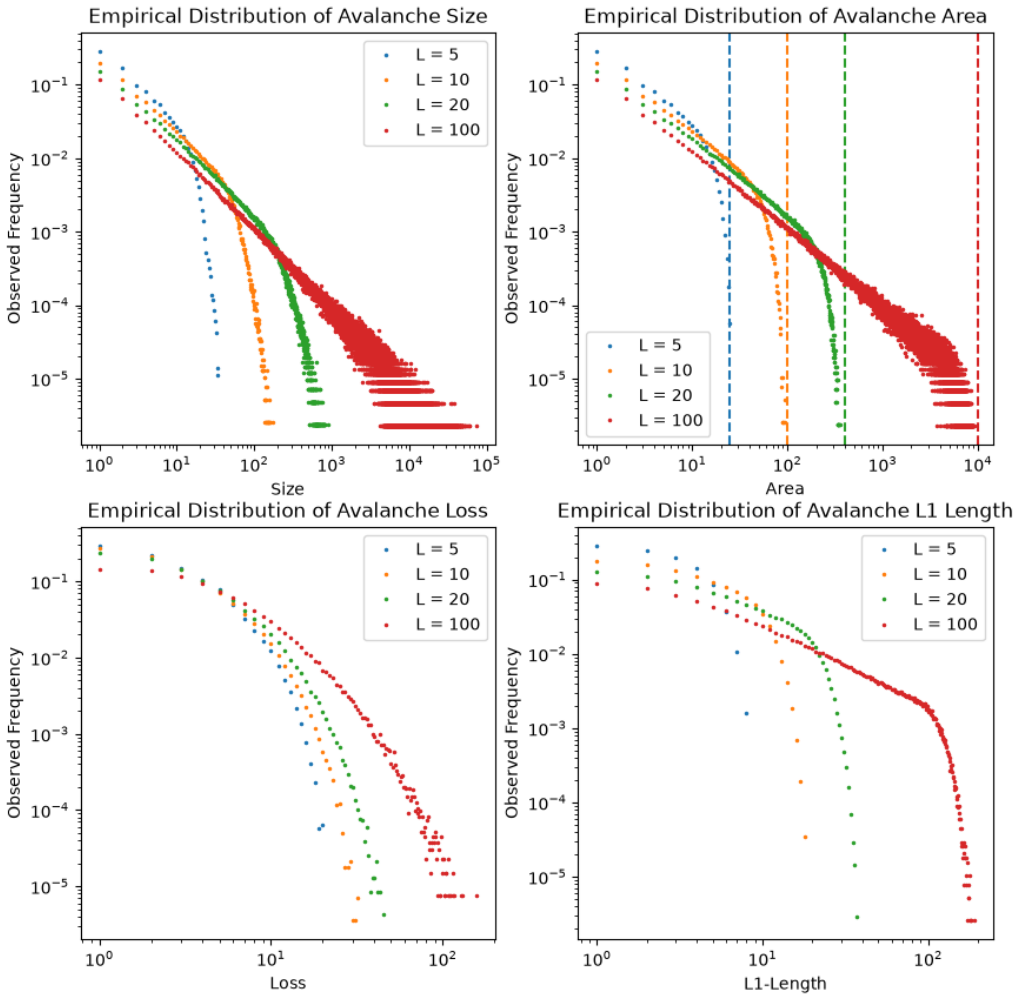


Figure 2.1: Observed frequencies of key avalanche statistics computed from the simulated sandpile Markov chain, excluding the discarded burn-in samples. A subset of four lattice sizes are depicted: 5, 10, 20, and 100, with varying colors. Values are computed from 10^6 simulated observations. The dashed vertical lines on the top right correspond to the maximum possible avalanche area for each lattice size, L^2. Plots are shown in log-log axes. The appearance of horizontal lines is an artifact of the log-log axes. The lines correspond to sizes with only 1, 2, 3, etc. observations in the dataset, with exaggerated vertical distance between the lines because of the log scale.

3

Power Law Fit

3.1 Bounded Power Law Fit

The linear component of the distribution observed in Figure 2.1 suggests fitting a power law distribution to the simulated sequence of avalanche sizes and areas.

We hypothesize a power law scaling relationship for Toppling Size S and area A of the form:

$$p(X = x; \tau, x_{max}) = C_{\tau, x_{max}} x^{-\tau} \text{ for } x \in \{1, \cdots, x_{max}\}$$

where $x_{max} = x_{max}(L)$ is a cutoff value reflecting the observed decay of the distribution away from power law scaling due to the finite lattice size L.

The shape of the pmf under varying parameter values can be observed in Figure 3.1.

To estimate the power law scaling coefficient τ, we apply the method of Maximum Likelihood as justified in Section 1.4.

Let $\mathbf{x} = (x_1, x_2, \cdots, x_n)$. The likelihood function is:

$$\mathcal{L}(\mathbf{x}; \tau, x_{max}) = \prod_{i=1}^{n} \frac{x_i^{-\tau}}{C_{\tau, x_{max}}}$$

$$\log \mathcal{L}(\mathbf{x}; \tau, x_{max}) = -n \log C_{\tau, x_{max}} - \tau \sum_{i=1}^{n} \log x_i$$

Where $C_{\tau, x_{max}}$ is the normalising constant:

$$C_{\tau, x_{max}} = \sum_{k=1}^{x_{max}} k^{-\tau}$$

To estimate the parameter τ assuming known x_{max}, we compute its Maximum Likelihood Estimate (MLE). This is the value $\hat{\tau}$ that satisfies:

$$\hat{\tau} = \arg\max_{\tau} \mathcal{L}(\mathbf{x}; \tau, x_{max}) = \arg\min_{\tau} \{-\log \mathcal{L}(\mathbf{x}; \tau, x_{max})\}$$

Because the normalising constant $C_{\tau, x_{max}}$ depends on the parameter τ, we cannot find an analytic solution for $\hat{\tau}$ and rely on numerical optimisation techniques. The numerical optimisation technique used here is the L-BFGS-B algorithm introduced by Byrd et al. (1995) [3] and implemented in the `scipy` python library.

Let $\tau = \tau_s(L), x_{max} = s_{max}(L)$ for modelling the avalanche size, $\tau = \tau_a(L), x_{max} = a_{max}(L)$ for modelling the avalanche area, and $\tau = \tau_d(L), x_{max} = d_{max}(L)$ for modelling the avalanche L_1 length. We first consider a full-data estimation procedure, where we ignore the non-linear distribution tails and fit τ for each statistic considered and lattice size simulated. Here, we fix $a_{max}(L) = L^2$, $d_{max}(L) = 2L$, and $s_{max}(L) = 10L^2$ as known; this is clearly justified as the theoretical maximum for avalanche area and L_1 length, and is a computational adjustment for avalanche size that exceeds any observed avalanche size in the simulated data.

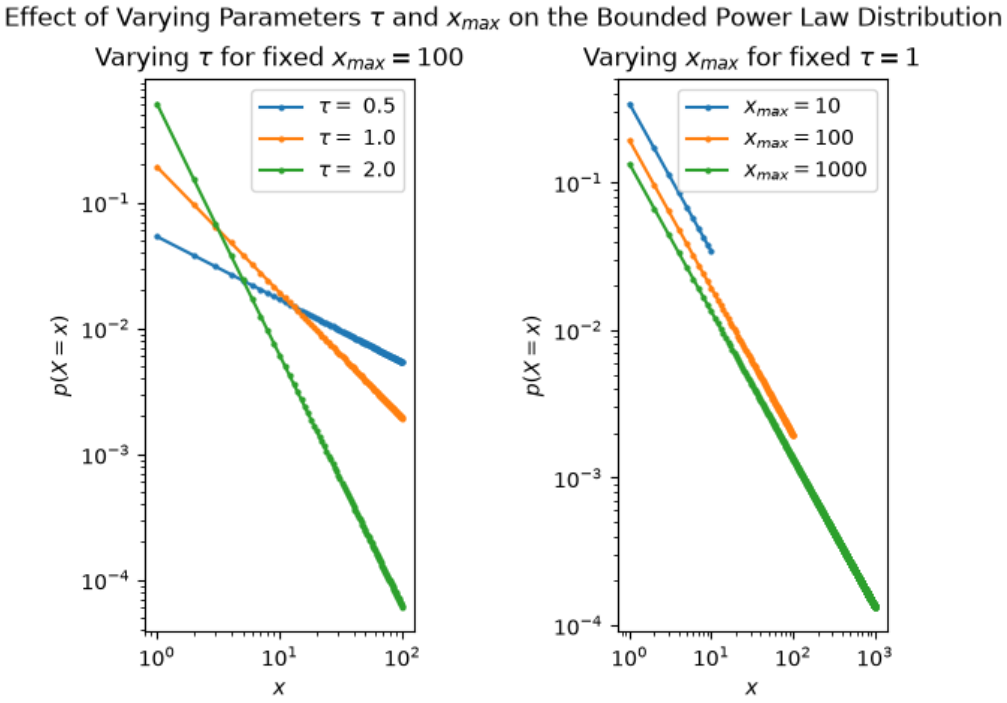


Figure 3.1: Visualisation of the bounded power law probability mass function on log-log axes under different parameter combinations. Each point represents a discrete value and corresponding probability mass, with a line connecting sequential points for illustration purposes. The left plot shows the effect of varying τ while keeping x_{max} constant, which results in changes to the slope and intercept of the connecting line. The right plot shows the effect of varying x_{max} while keeping τ constant, which results in vertical translation of the connecting line, with different support for x. The orange line is identical in both plots, corresponding to parameters $\tau = 1$, $x_{max} = 100$.

Under the finite size scaling hypothesis discussed in 1.4, we expect that the power law coefficient should be constant for all values of L, i.e. we expect $\tau_s(L) = \tau_s$, $\tau_a(L) = \tau_a$, and $\tau_d(L) = \tau_d$.

The estimates obtained are presented in Figure 3.2 and Table 3.1. We observe a decreasing trend in the estimated power law coefficients for both avalanche size and avalanche area. However, in Figure 3.3 we see that the tail has a substantial effect on the fit, creating an artificially high estimate for the slope. This effect appears most pronounced at smaller lattice sizes, but is still evident at $L = 100$, suggesting the decrease in $\hat{\tau}$ could be caused by poor fit at the tails.

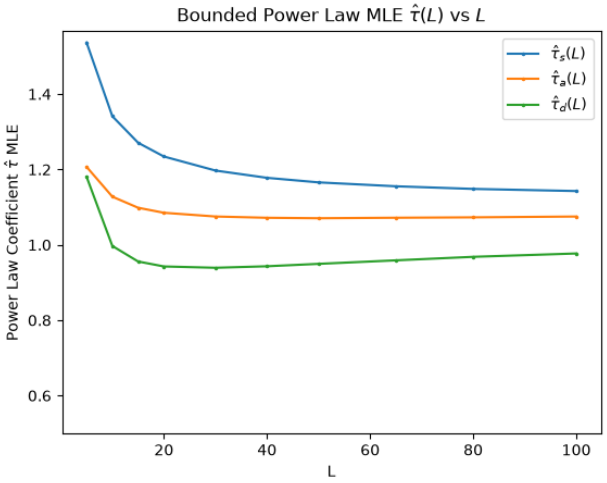


Figure 3.2: Estimates for $\hat{\tau}_s(L), \hat{\tau}_a(L), \hat{\tau}_d(L)$.

L	$\hat{\tau}_s(L)$	$\hat{\tau}_a(L)$	$\hat{\tau}_d(L)$
5	1.54	1.21	1.18
10	1.34	1.13	1.00
15	1.27	1.10	0.96
20	1.23	1.09	0.94
30	1.20	1.08	0.94
40	1.18	1.07	0.94
50	1.17	1.07	0.95
65	1.16	1.07	0.96
80	1.15	1.07	0.97
100	1.14	1.08	0.98

Table 3.1: Bounded Power Law MLE coefficients $hat\tau$ obtained for varying lattice sizes L

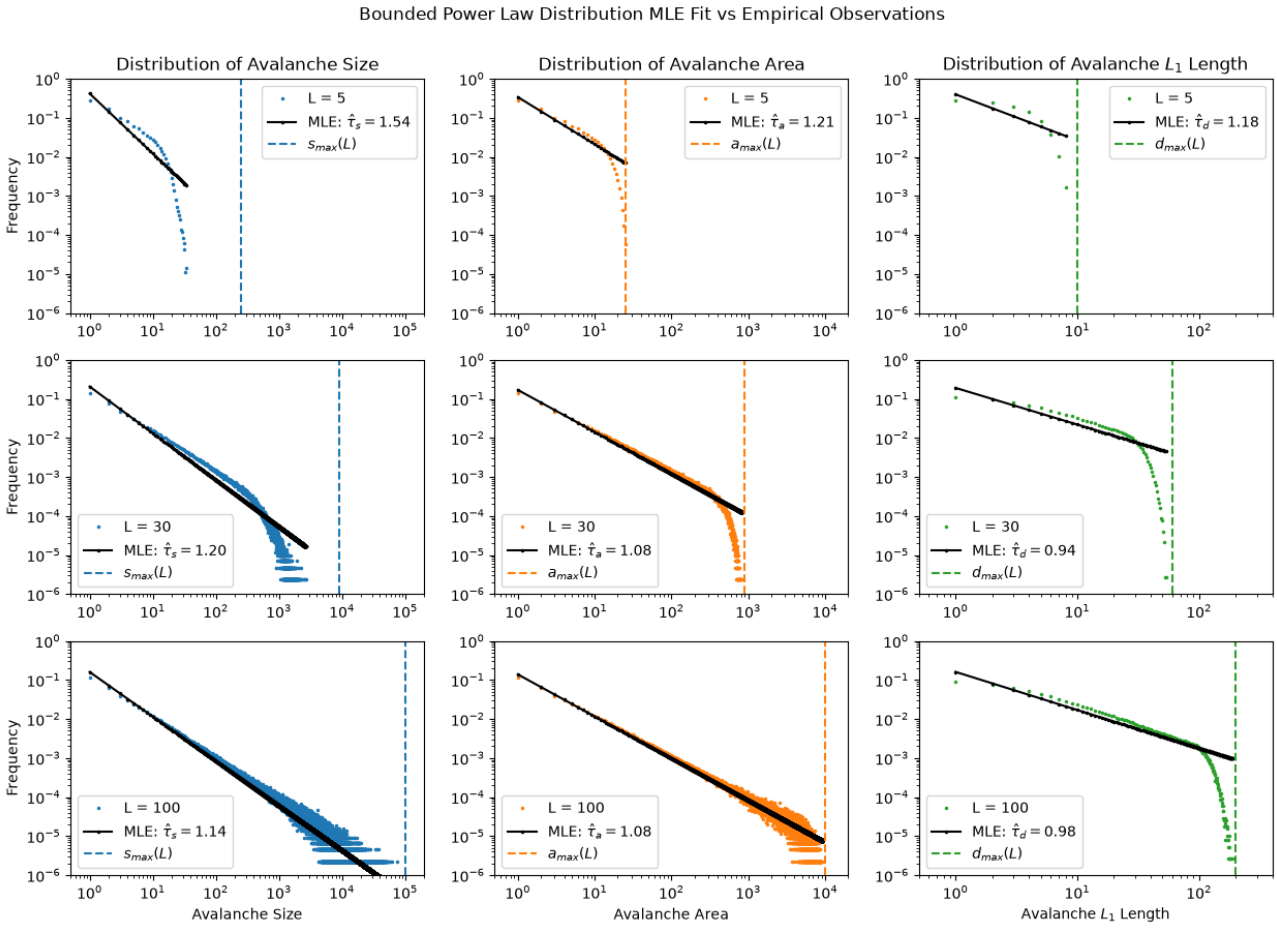


Figure 3.3: Fitted MLEs for the bounded power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure A.1. MLEs are drawn as black points with a connecting line drawn for illustration purposes. The left column shows avalanche size, while the right column shows avalanche area. The dashed vertical lines represent the fixed parameter x_{max}.

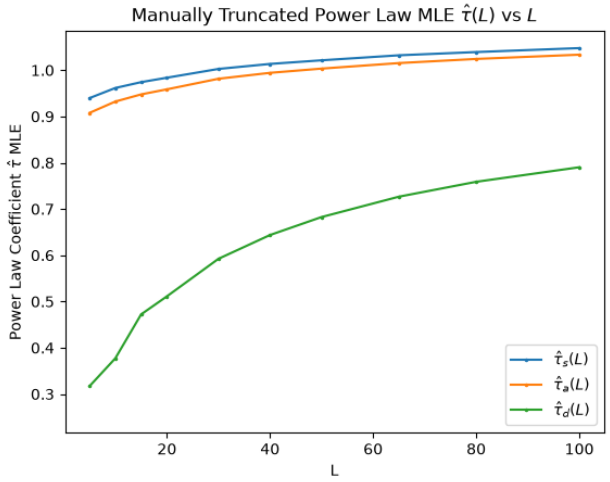


Figure 3.4: Estimates for $\hat{\tau}_s(L), \hat{\tau}_a(L), \hat{\tau}_d(L)$ under the manual truncation rule $s_{max} = 0.2L^2, a_{max} = 0.1L^2, d_{max} = 0.5L$.

L	$\hat{\tau}_s(L)$	$\hat{\tau}_a(L)$	$\hat{\tau}_d(L)$
5	0.94	0.91	0.32
10	0.96	0.93	0.38
15	0.97	0.95	0.47
20	0.98	0.96	0.51
30	1.00	0.98	0.59
40	1.01	0.99	0.64
50	1.02	1.00	0.68
65	1.03	1.02	0.73
80	1.04	1.02	0.76
100	1.05	1.03	0.79

Table 3.2: MLE power-law slopes obtained for varying lattice sizes L under the manual truncation rule $s_{max} = 0.2L^2, a_{max} = 0.1L^2, d_{max} = 0.5L$

3.2 Manually-truncated Power Law

As a first correction for the finite lattice effect, we consider restricting $s_{max}, a_{max}, d_{max}$ to include only those topple sizes, areas, and lengths for which linearity is observed in the log-log plot. As we saw in Figure 3.1, changing x_{max} should not affect the estimated slope of the distribution.

To avoid manually setting an arbitrary maximum for each value of L, here we consider the rule-of-thumb approximation $s_{max}0.2L^2, a_{max} = 0.1L^2, d_{max} = 0.5L$. Clearly, there are significant limitations to this approach: the cutoff rule was chosen arbitrarily, and data is significantly limited for lower values of L. However this will help us get a more accurate idea of the power law structure.

The results in Table 3.2 and Figure 3.4 are noticeably lower than the previous estimates in Table 3.1, with an increasing trend rather than decreasing. From these two analyses, it is reasonable that the powerlaw-governed section of the distribution has exponent $\tau_s, \tau_a \approx 1$, which is consistent with the estimate presented in the original 1987 paper by Bak et al. [1].

To avoid discarding data, we turn to explicit models of the tail distribution, which we use to attempt to capture the effect of the finite lattice size L and obtain a more informed approximation to the powerlaw scaling coefficient.

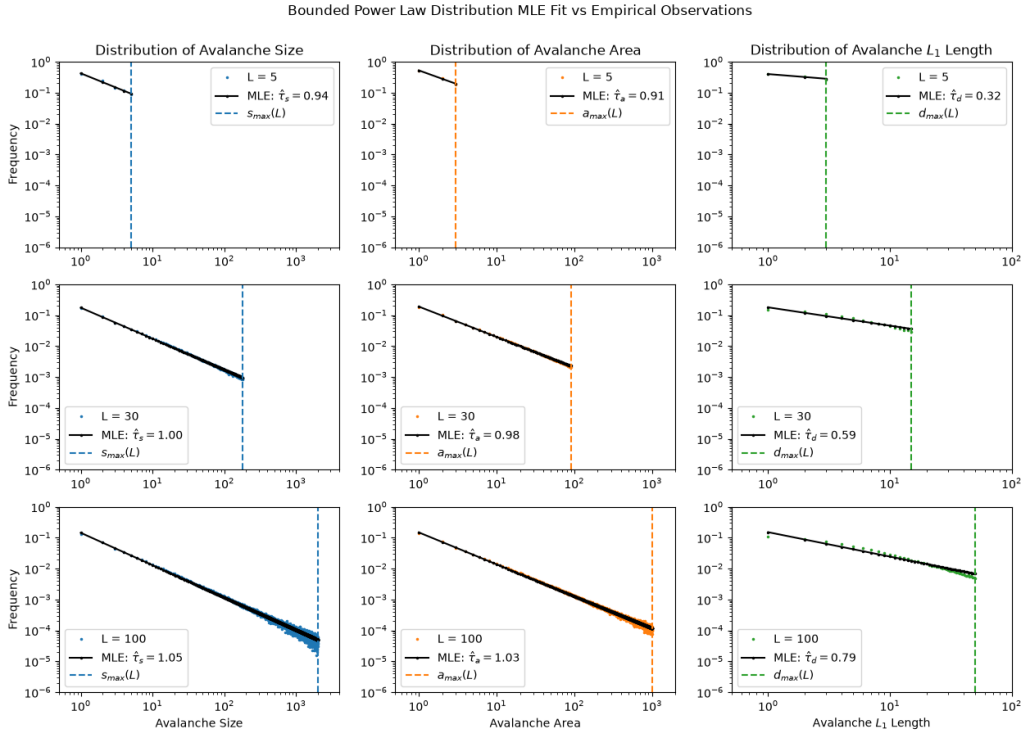


Figure 3.5: Fitted MLEs for the manually-truncated power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure A.2. MLEs are drawn as black points with a connecting line drawn for illustration purposes. The left column shows avalanche size, while the right column shows avalanche area. The dashed vertical lines on the right show the manual truncation rule bounds $s_{max} = 0.2L^2, a_{max} = 0.1L^2, d_{max} = 0.5L$.

4

Finite Size Scaling Effect

Recalling Section 1.4, the finite size scaling law hypothesizes that the tail behaviour relates to the finite lattice size L via a probability mass function of the form:

$$p(X = x; \tau, L, f, D) \propto x^{-\tau} f\left(\frac{x}{L^D}\right)$$

Here f governs the tail behaviour of the distribution, i.e. for small values of u, $f(u) \approx 1$ and the distribution resembles a power law.

4.1 Exponentially Truncated Power Law

We first consider the exponentially-truncated power law:

$$p(X = x; \tau, c, x_{max}) = \frac{1}{C_{\tau,c,x_{max}}} x^{-\tau} \exp\left(\frac{-x}{c}\right)$$

Where $C_{\tau,c,x_{max}}$ is the normalising constant:

$$C_{\tau,c,x_{max}} = \sum_{k=1}^{x_{max}} k^{-\tau} \exp\left(-k/c\right)$$

In the above pmf, τ and x_{max} are, as before, the power law scaling coefficient and the support upper bound respectively, while the new parameter c controls the location of the soft cutoff, where power law behaviour decays.

The shape of the pmf under varying parameter values can be observed in Figure 4.1

The likelihood and log-likelihood are:

$$\mathcal{L}(\mathbf{x}; \tau, c, x_{max}) = \prod_{i=1}^{n} \frac{1}{C_{\tau,c,x_{max}}} x_i^{-\tau} \exp\left(\frac{-x_i}{c}\right)$$

$$\log \mathcal{L}(\mathbf{x}; \tau, c, x_{max}) = -n \log C_{\tau,c,x_{max}} - \tau \sum_{i=1}^{T} \log x_i - \frac{1}{c} \sum_{i=1}^{T} x_i$$

As before, for each L, we numerically optimise the log-likelihood function to obtain:

$$\hat{\tau}(L), \hat{c}(L) = \arg\min_{\tau,c} \left\{-\log \mathcal{L}(\mathbf{x}_L; \tau, c, x_{max})\right\}$$

Set $\tau = \tau_s(L), x_{max} = s_{max}(L), c = c_s(L)$ for modelling the avalanche size, $\tau = \tau_a(L), x_{max} = a_{max}(L), c = c_a(L)$, and $\tau = \tau_d(L), x_{max} = d_{max}(L), c = c_d(L)$ for modelling the avalanche area. Recall we fix $a_{max}(L) = L^2$, $d_{max} = 2L$, and $s_{max}(L) = 10L^2$ as known.

The results in Table 4.1 are promising: for lattice sizes 40 and above, the powerlaw coefficient $\tau \approx 1$. Further, in Figure 4.2 the log soft cutoffs $\log \hat{c}$ appear to scale linearly in $\log L$, a relationship that we will examine in more detail when computing the pooled estimators.

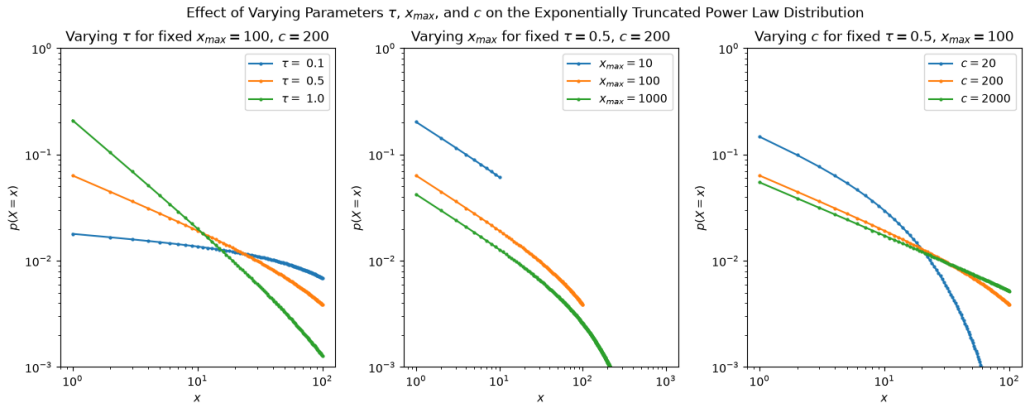


Figure 4.1: Visualisation of the exponentially truncated power law probability mass function on log-log axes under different parameter combinations. Each point represents a discrete value and corresponding probability mass, with a line connecting sequential points for illustration purposes. The left plot shows the effect of varying τ while keeping x_{max} and c constant, which results in changes to the slope of the linear part of the connecting line. The central plot shows the effect of varying x_{max} while keeping τ and c constant, which results in vertical translation of the connecting curve, with different support for x. The right plot shows the effect of increasing c while keeping τ and x_{max} constant, which results in later decay.The orange line is identical in all plots, corresponding to parameters $\tau = 0.5$, $x_{max} = 100$, and $c = 200$.

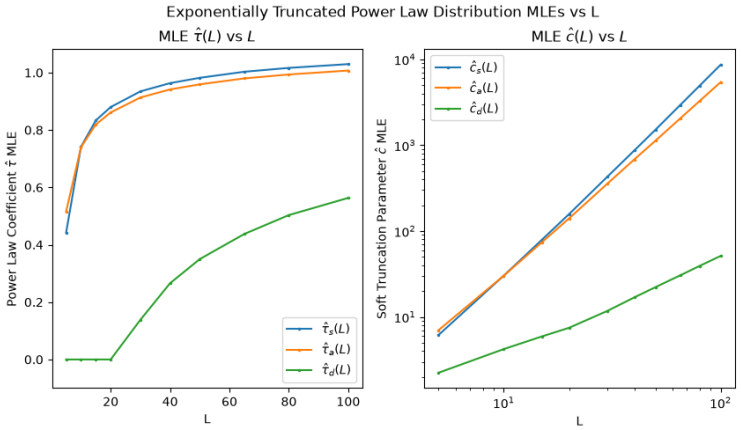


Figure 4.2: Estimates MLEs $\hat{\tau}_s(L), \hat{\tau}_a(L), \hat{\tau}_d(L)$ (left) and $\hat{c}_s(L), \hat{c}_a(L), \hat{c}_d(L)$ (right) for varying lattice sizes L.

L	$\hat{\tau}_s(L)$	$\hat{c}_s(L)$	$\hat{\tau}_a(L)$	$\hat{c}_a(L)$	$\hat{\tau}_d(L)$	$\hat{c}_d(L)$
5	0.44	6.15	0.52	7.01	0.00	2.24
10	0.74	30.26	0.74	30.24	0.00	4.22
15	0.83	79.33	0.82	74.17	0.00	5.93
20	0.88	157.56	0.86	140.37	0.00	7.47
30	0.93	428.18	0.91	355.29	0.14	11.73
40	0.96	868.82	0.94	683.27	0.27	16.88
50	0.98	1509.00	0.96	1124.60	0.35	22.16
65	1.00	2924.10	0.98	2041.31	0.44	30.43
80	1.02	4932.09	0.99	3269.35	0.50	39.24
100	1.03	8647.45	1.01	5427.84	0.56	51.29

Table 4.1: MLE power-law slopes τ and soft cutoffs c obtained for varying lattice sizes L

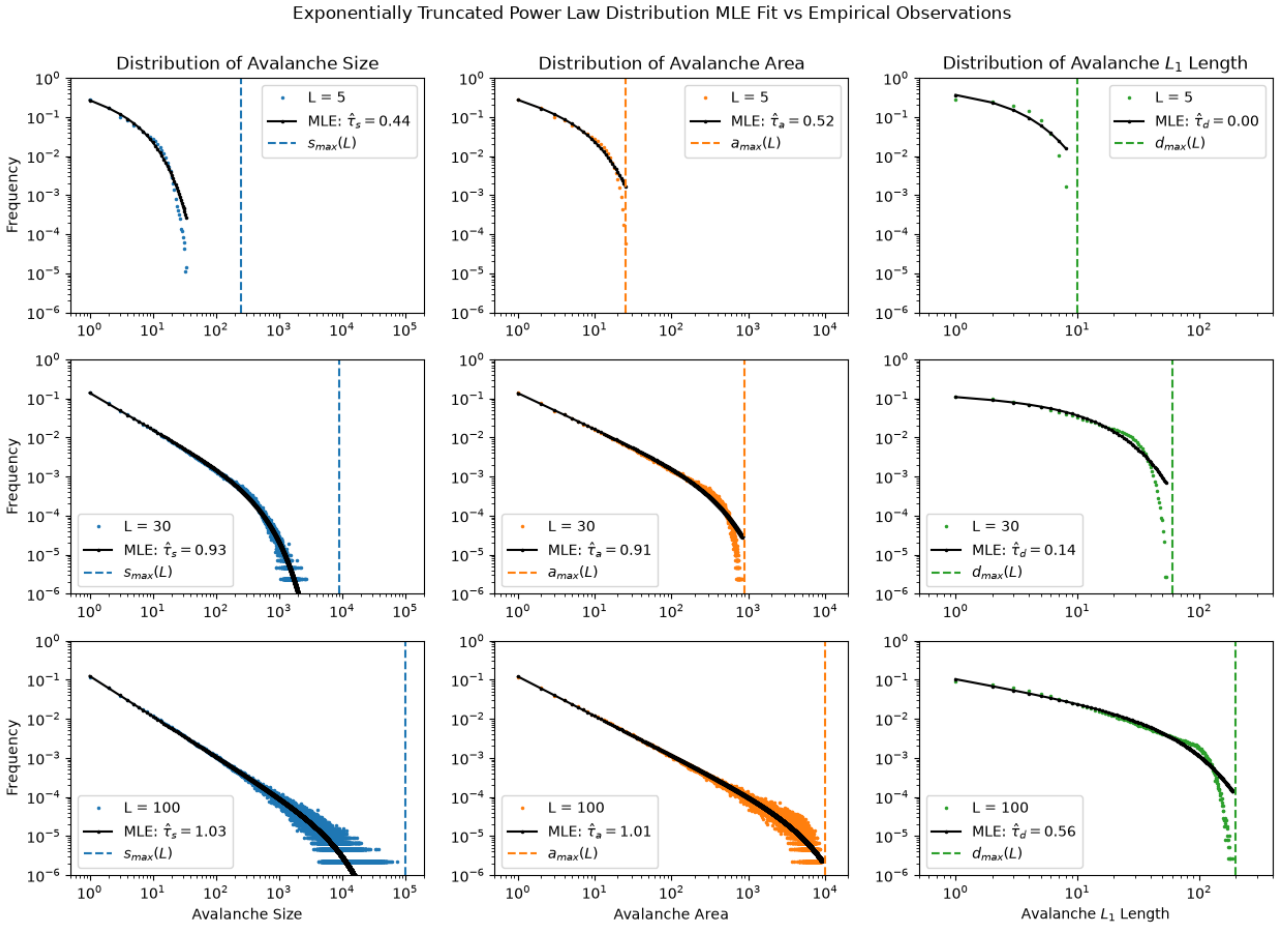


Figure 4.3: Fitted MLEs for the unpooled exponentially truncated power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure A.3. MLEs are drawn as black points with a connecting line drawn for illustration purposes. The left column shows avalanche size, while the right column shows avalanche area. The dashed vertical lines on the right show the bound on avalanche area, L^2.

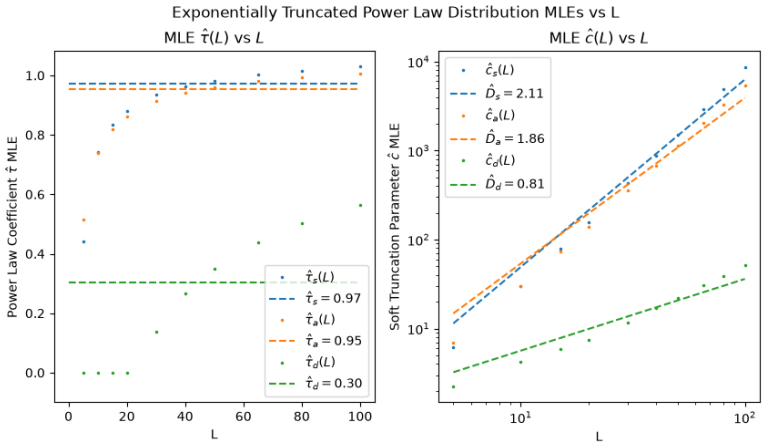


Figure 4.4: A comparison of the unpooled estimates of the parameters (points) with the pooled parameter estimates (dashed lines). If the pmf form and the finite size scaling hypothesis are appropriate, we expect the dashed line to approximate the trend of the points. Based on the above, the model is not quite appropriate, and we need to consider an additional model for the pmf.

	$\hat{\tau}$	$\hat{k}$	$\hat{D}$
Size s	0.97	0.38	2.11
Area a	0.95	0.74	1.86
L_1 Length d	0.30	0.88	0.81

Table 4.2: Pooled MLE coefficients for finite size scaling assuming the exponentially truncated power law distribution. We expect $\hat{D} \approx 2$ and $\hat{\tau} \approx 1$ for both avalanche area and size, so the results seem reasonable.

4.2 Pooled Exponentially Truncated Power Law

Consider again the Finite Size Scaling hypothesis:

$$p(X = x; \tau, L, f, D) \propto x^{-\tau} f\left(\frac{x}{L^D}\right)$$

In the previous section, we modelled

$$p(X = x; \tau, L, c, D) \propto x^{-\tau} \exp\left(-\frac{x}{c}\right)$$

Therefore, if the chosen pmf is adequate to model the behaviour of the system, and the statistics obey finite size scaling, then we should observe a constant powerlaw coefficient $\tau_s = \tau_s(L)$, $\tau_a = \tau_a(L)$, $\tau_d = \tau_d(L)$ as well as the relationships $c_s(L) = k_s L_s^D$, $c_a(L) = k_a L_a^D$, and $c_d(L) = k_d L_d^D$ for some constants $k_s, k_a, k_d, D_s, D_a, D_d$.

To assess whether this is plausible, consider the pooled likelihood. Let $\mathbf{L} = (L_1, L_2, \cdots, L_K)$ be the set of lattice sizes simulated, and let $\mathbf{x}^j = (x_1^j, \cdots, x_n^j)$ be the set of simulated statistics from lattice size L_j.

$$\mathcal{L}(\mathbf{x}_1, \cdots \mathbf{x}_K; \mathbf{L}, \tau, k, D) = \prod_{j=1}^{K} \mathcal{L}(\mathbf{x}_j; \tau, c = kL_j^D, x_{max} = x_{max}(L_j))$$

The full dataset can be used to fit τ, k, and D using maximum likelihood, and the fit can be compared against the individual fits. If the pmf is well-specified and the system obeys finite size scaling, the fits should not significantly degrade, and we should observe that $\log \hat{c} = \log \hat{k} + \hat{D} \log L$. In other words, the pooled fit should accurately compute the trendline on the graph of $\log \hat{c}$ vs $\log L$. While the estimates in Table 4.2 appear reasonable, the results in Figure 4.4 show that the pooled parameters do not fully capture the behaviour of the pmf. In particular, $\hat{\tau}$ does not appear to be constant for either avalanche size nor area, and the fitted line on the graph of $\log \hat{c}$ vs $\log L$ is not a great fit. This suggests the estimates of $\hat{k}$ and $\hat{D}$ are smoothing over the excess error introduced by assuming a constant $\hat{\tau}$.

4.3 Stretched Exponentially Truncated Power Law

We extend the exponentially truncated power law studied above to allow more flexibility in the tails. We consider both the unpooled and pooled models.

The pmf is:

$$p(X = x; \tau, c, \beta, x_{max}) = \frac{1}{C_{\tau,c,\beta,x_{max}}} x^{-\tau} \exp\left(-\left(\frac{x}{c}\right)^{\beta}\right)$$

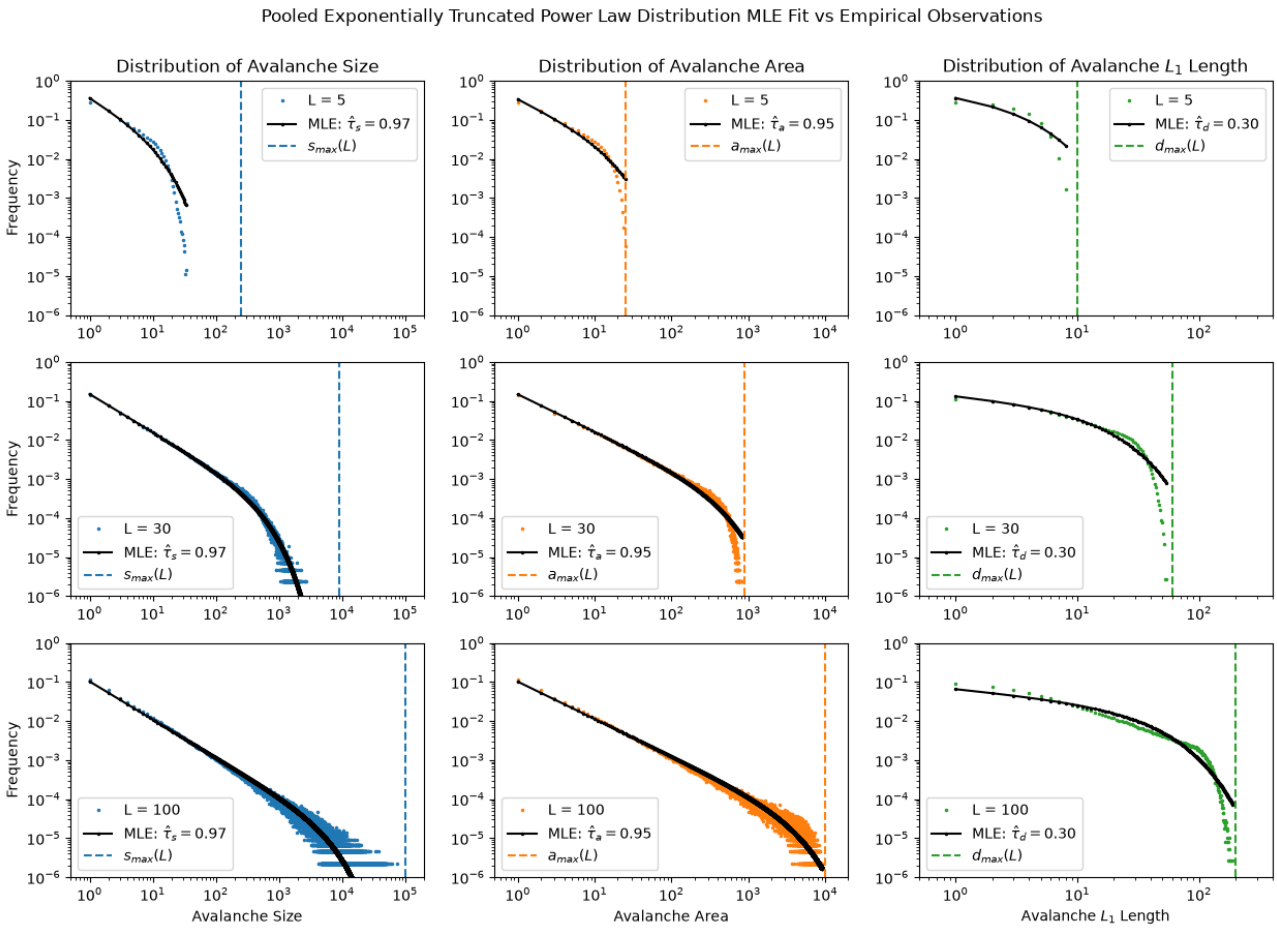


Figure 4.5: Fitted MLEs for the pooled exponentially truncated power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure A.4. MLEs are drawn as black points with a connecting line drawn for illustration purposes.

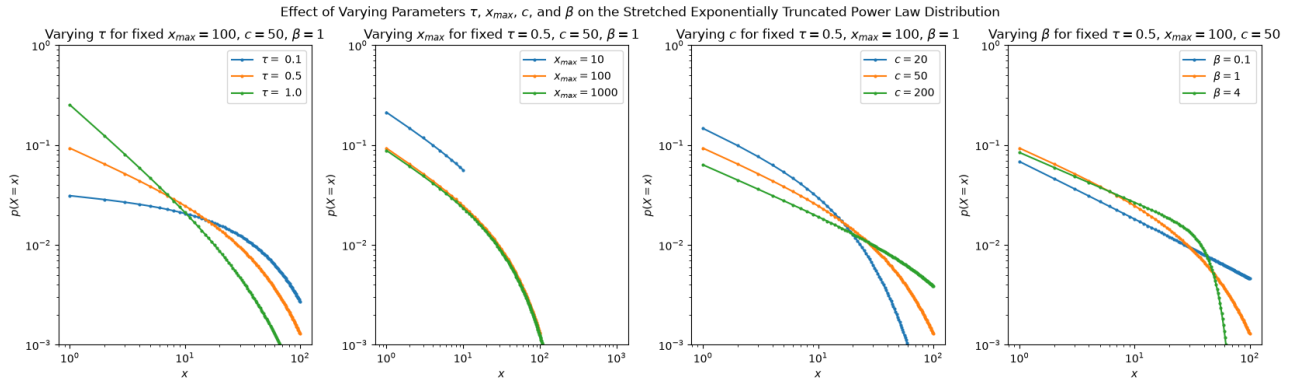


Figure 4.6: Visualisation of the stretched exponentially truncated power law probability mass function on log-log axes under different parameter combinations. Each point represents a discrete value and corresponding probability mass, with a line connecting sequential points for illustration purposes. The left three plots are identical to Figure 4.1 as they keep $\beta = 1$. The right plot shows the effect of increasing β while keeping τ, x_{max}, and c constant, which results in steeper decay.The orange line is identical in all plots, corresponding to parameters $\tau = 0.5$, $x_{max} = 100$, $c = 200$, and $\beta = 1$.

Where $C_{\tau,c,\beta,x_{max}}$ is the normalising constant:

$$C_{\tau,c,x_{max}} = \sum_{k=1}^{x_{max}} k^{-\tau} \exp\left(-(k/c)^{\beta}\right)$$

The shape of the pmf under varying parameter values can be observed in Figure 4.6. Of particular interest is that β can impose a rapid transition from linearity to decay in the log-log plot.

The unpooled likelihood and log-likelihood are:

$$\mathcal{L}(\mathbf{x};\tau,c,x_{max},\beta) = \prod_{i=1}^{n} \frac{1}{C_{\tau,c,\beta,x_{max}}} x_i^{-\tau} \exp\left(-\left(\frac{x_i}{c}\right)^{\beta}\right)$$

$$\log \mathcal{L}(\mathbf{x};\tau,c,\beta,x_{max}) = -n \log C_{\tau,c,\beta,x_{max}} - \tau \sum_{i=1}^{T} \log x_i - \frac{1}{c^{\beta}} \sum_{i=1}^{T} x_i^{\beta}$$

As before, we can extend this model under the finite size scaling hypothesis to be of the form

$$p(X = x;\tau,L,k,D,\beta) \propto x^{-\tau} \exp\left(-\left(\frac{x}{kL^D}\right)^{\beta}\right)$$

And we assume $\tau_s = \tau_s(L)$, $\tau_a = \tau_a(L)$, $\beta_s = \beta_s(L)$, $\beta_a = \beta_a(L)$ as well as the relationship $c_s(L) = k_s L_s^D$ and $c_a(L) = k_a L_a^D$ for some constants k_s, k_a, D_s, D_a.

The pooled likelihood is:

$$\mathcal{L}(\mathbf{x}_1,\cdots \mathbf{x}_K;\mathbf{L},\tau,k,D,\beta) = \prod_{j=1}^{K} \mathcal{L}(\mathbf{x}_j;\tau,c = kL_j^D,\beta x_{max} = x_{max}(L_j))$$

The unpooled fits in Figure 4.7 appear to model the pmfs very closely, capturing the strong decay associated with small L and the area hard cutoff L^2.

The pooled fit reveals a very interesting difference between avalanche size and avalanche area. Examining Figure 4.8, while avalanche area could plausibly have $\beta_a(L) = \beta_a$, β_s has a clear L-dependence. As L increases, the soft cutoff c increases, but the steepness of the decay β decreases. This makes sense, as repeated toppling of the same locations are interrupted by loss at the boundaries, but when the lattice size increases, more grains are retained in the interior for longer, allowing repeated toppling. This is not the case for area; for any L, the avalanche area cannot exceed L^2, and therefore a sharp decay is needed near the cutoff point.

L	$\hat{\tau}_s(L)$	$\hat{c}_s(L)$	$\hat{\beta}_s(L)$	$\hat{\tau}_a(L)$	$\hat{c}_a(L)$	$\hat{\beta}_a(L)$	$\hat{\tau}_d(L)$	$\hat{c}_d(L)$	$\hat{\beta}_d(L)$
5	0.93	16.35	2.89	0.95	16.63	3.74	0.20	5.15	3.54
10	0.94	58.46	2.20	0.96	57.94	3.68	0.35	10.19	3.31
15	0.95	132.94	1.89	0.97	127.99	3.70	0.44	15.36	3.26
20	0.96	240.46	1.67	0.98	227.45	3.70	0.50	20.50	3.18
30	0.98	585.36	1.47	1.00	523.71	3.84	0.58	31.09	3.15
40	0.99	1105.57	1.35	1.01	949.78	3.89	0.64	41.66	3.07
50	1.00	1801.78	1.24	1.02	1502.50	3.91	0.67	52.02	2.99
65	1.01	3245.28	1.13	1.03	2594.20	3.95	0.71	67.22	2.83
80	1.02	5202.40	1.06	1.03	3992.73	3.95	0.75	83.47	2.83
100	1.03	8802.59	1.02	1.04	6348.53	4.11	0.77	103.50	2.66

Table 4.3: Unpooled MLE power-law slopes τ, soft cutoffs c, and stretching coefficient β obtained for varying lattice sizes L.

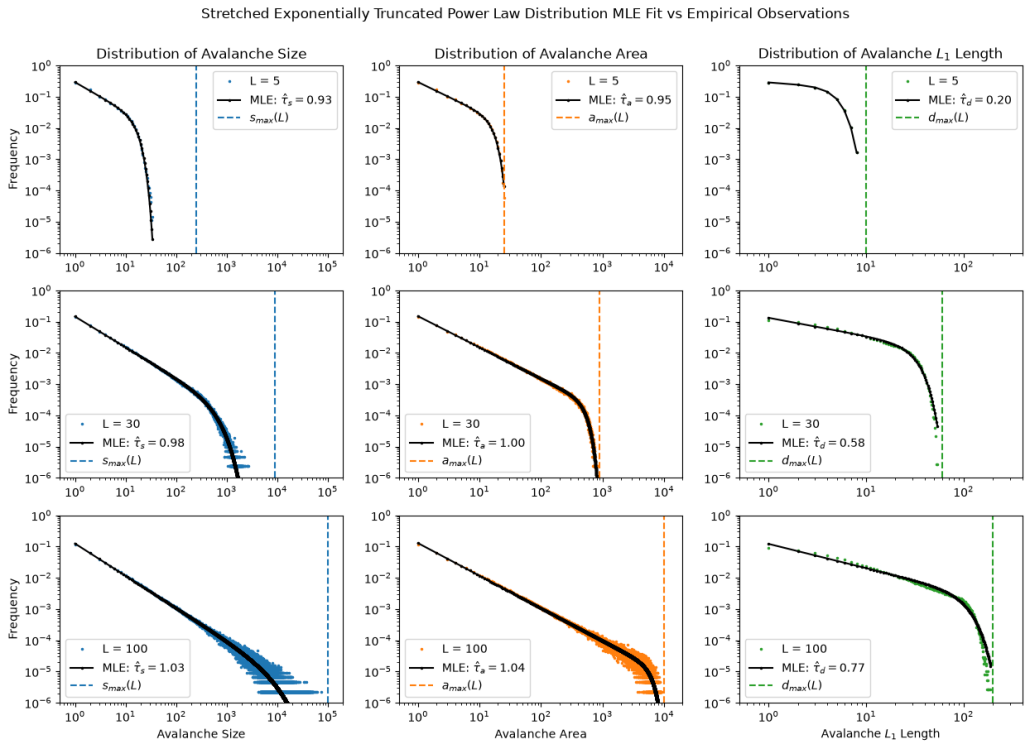


Figure 4.7: Fitted MLEs for the unpooled stretched exponentially truncated power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure A.5. MLEs are drawn as black points with a connecting line drawn for illustration purposes. The left column shows avalanche size, while the right column shows avalanche area. The dashed vertical lines on the right show the bound on avalanche area, L^2. The additional flexibility of the stretching parameter is able to capture the tail behaviour of the pmfs.

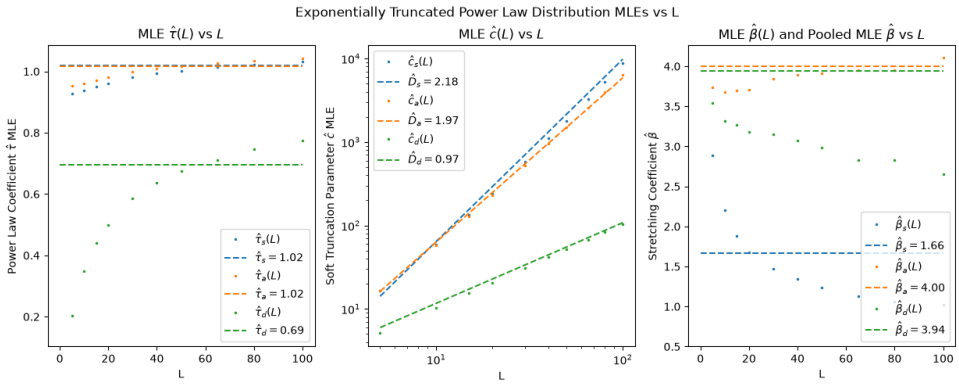


Figure 4.8: A comparison of the unpooled estimates of the parameters (points) with the pooled parameter estimates (dashed lines). If the pmf form and the finite size scaling hypothesis are appropriate, we expect the dashed line to approximate the trend of the points. Based on the above, the pooled model for avalanche area is reasonable, while evidence suggests avalanche size may have an $L-$dependent stretching parameter β_s.

	$\hat{\tau}$	$\hat{k}$	$\hat{D}$	$\hat{\beta}$
Size s	1.02	0.42	2.18	1.66
Area a	1.02	0.69	1.97	4.00
L_1 Length d	0.69	1.26	0.97	3.94

Table 4.4: Pooled MLE coefficients for finite size scaling assuming the stretched exponentially truncated power law distribution. We expect $\hat{D} \approx 2$ and $\hat{\tau} \approx 1$ for both avalanche area and size, so the results seem reasonable.

This distinction is clear in the plots of $\log \hat{c}$ vs $\log L$ in Figure 4.8 as well. While the avalanche area shows a well-fitted trendline for the pooled estimators, the avalanche size line does not appear to fit the estimates as well and is slightly offset from the trend in the unpooled estimators. This is because the estimates for k and D adapt to reduce the error of setting a fixed β_s. Because area can plausibly have a fixed β_a, this effect is minimised.

This is consistent with previous work [**?**], which found that while avalanche area obeys finite size scaling, toppling size has multifractal scaling and is not consistent with finite size scaling.

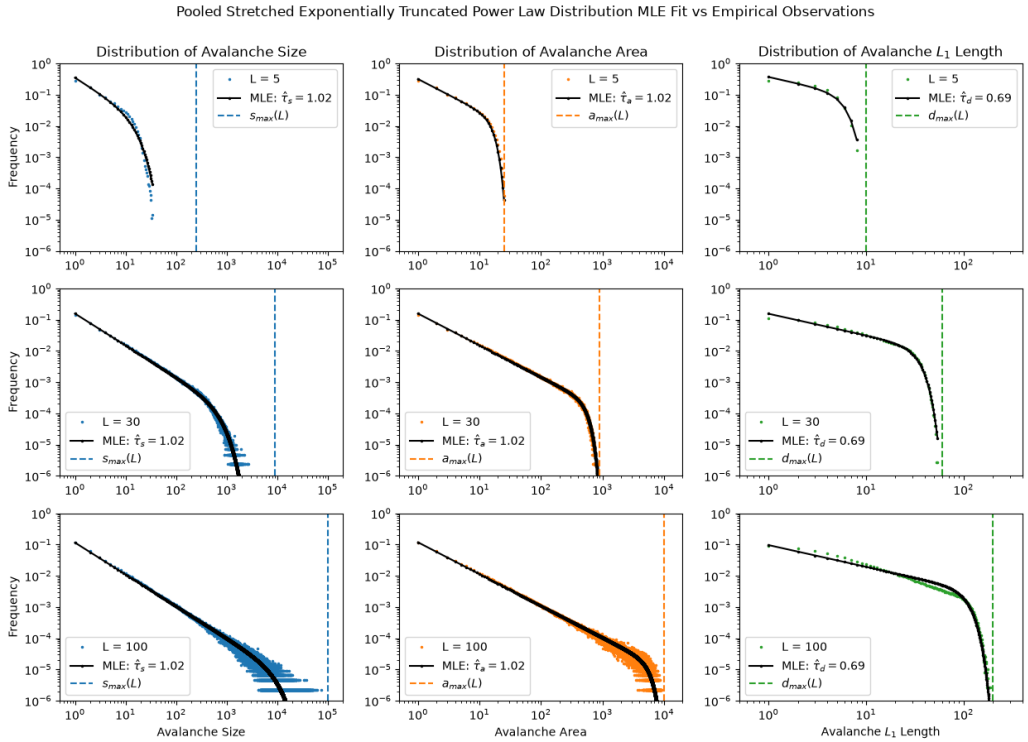


Figure 4.9: Fitted MLEs for the pooled stretched exponentially truncated power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure A.6. MLEs are drawn as black points with a connecting line drawn for illustration purposes. The left column shows avalanche size, while the right column shows avalanche area. The dashed vertical lines on the right show the bound on avalanche area, L^2. The pooled fit degrades compared to the unpooled fit in Figure 4.7 for the avalanche size. This is consistent with avalanche size not following a finite size scaling rule. However, the fit is reasonable for avalanche area.

5

Conclusion

5.1 Reflection

Evidence of problem solving and creativity

Evidence that student has extended their knowledge

Shows initiative

Appendix A

Extended MLE Fit Plots

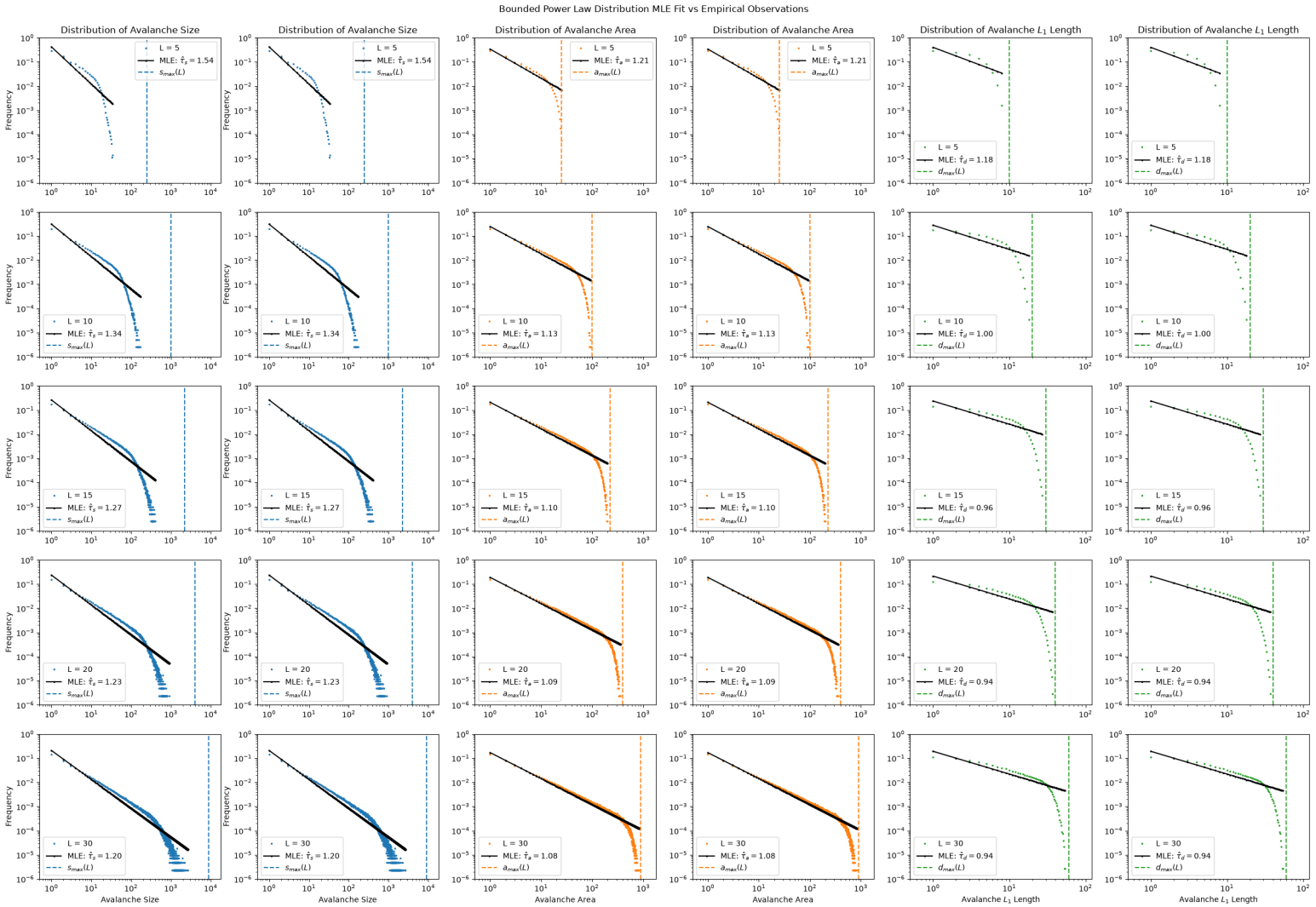


Figure A.1: Extended version of Figure 3.3 for all simulated lattice sizes.

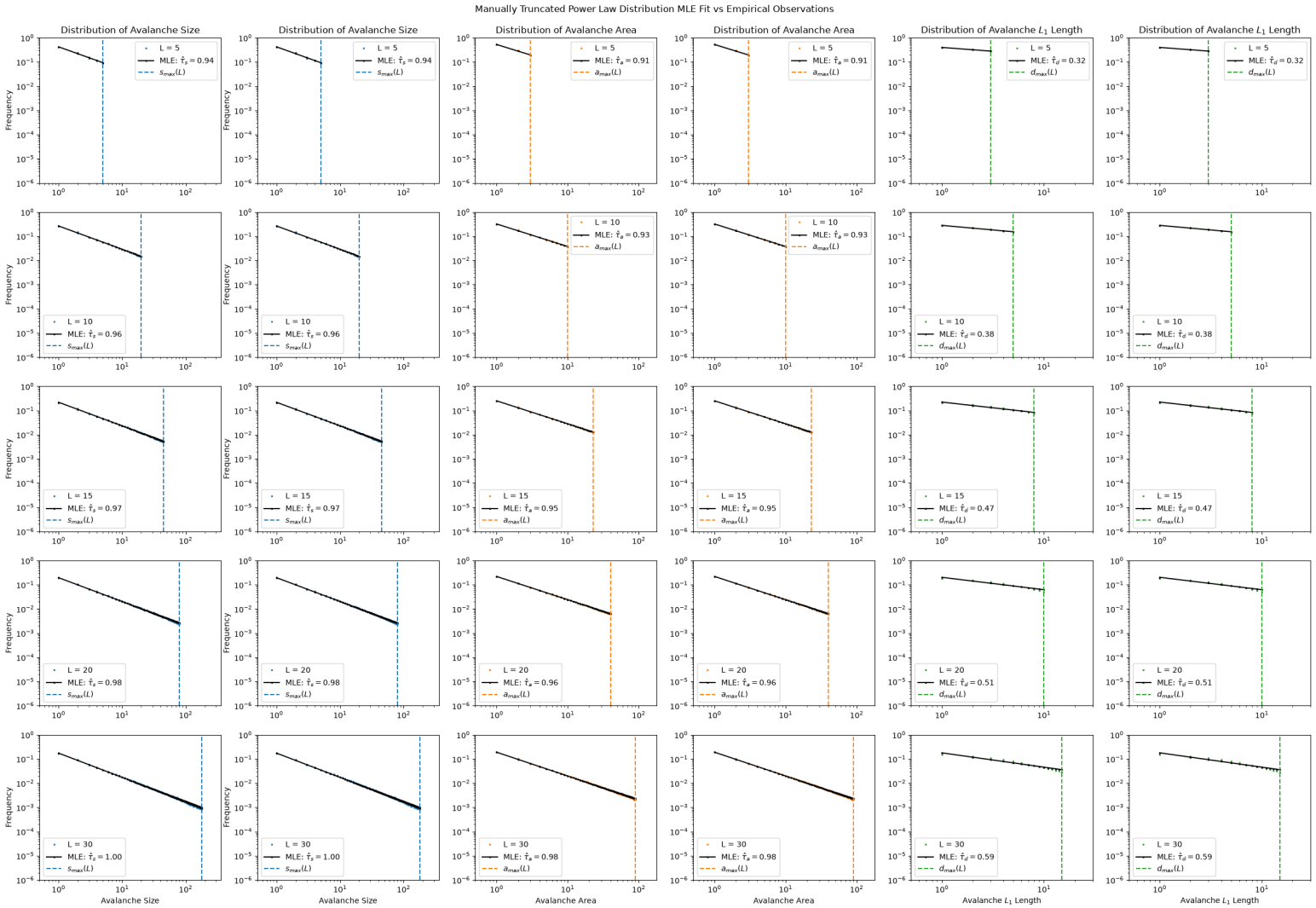


Figure A.2: Extended version of Figure 3.5 for all simulated lattice sizes.

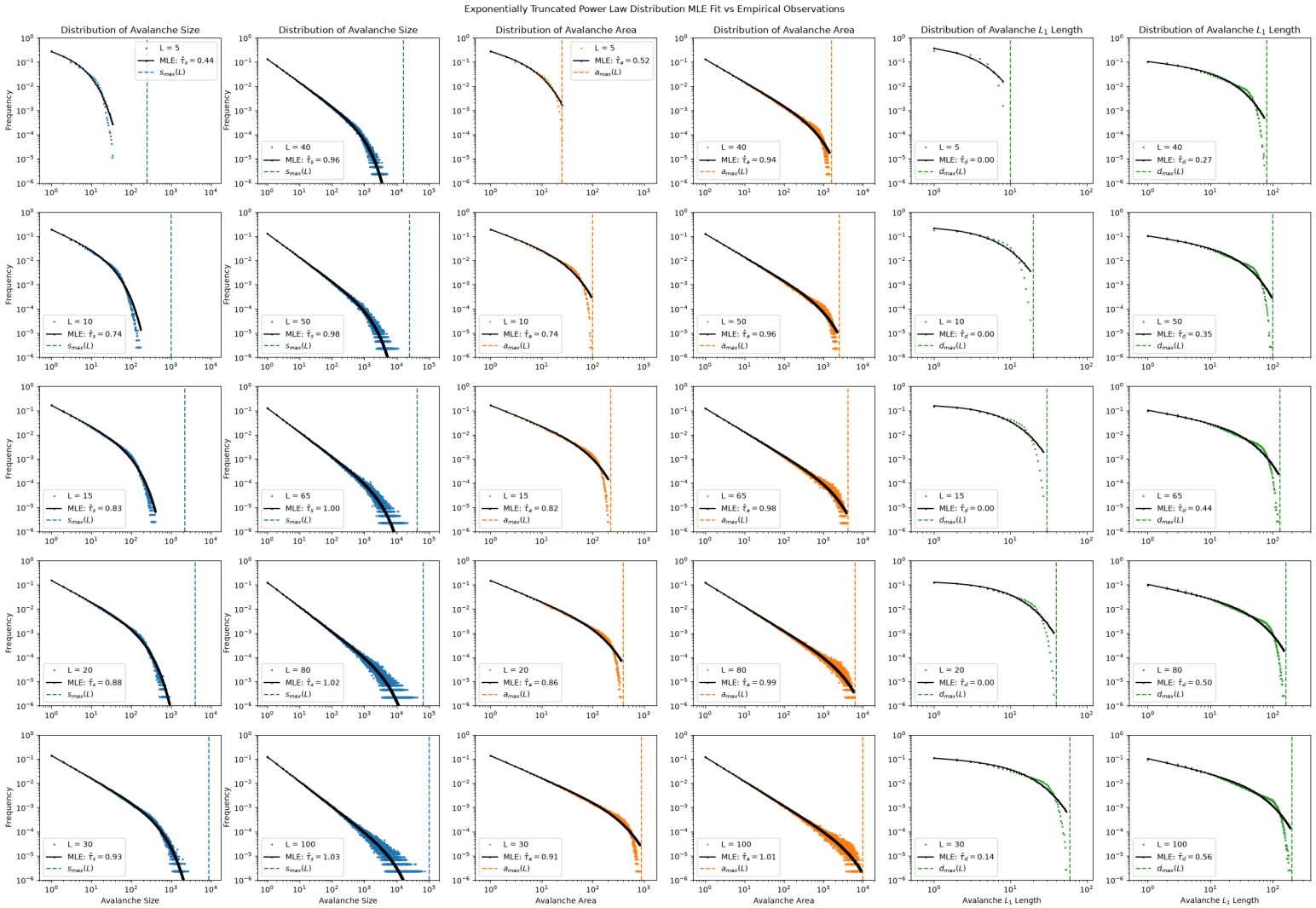


Figure A.3: Extended version of Figure 4.3 for all simulated lattice sizes.

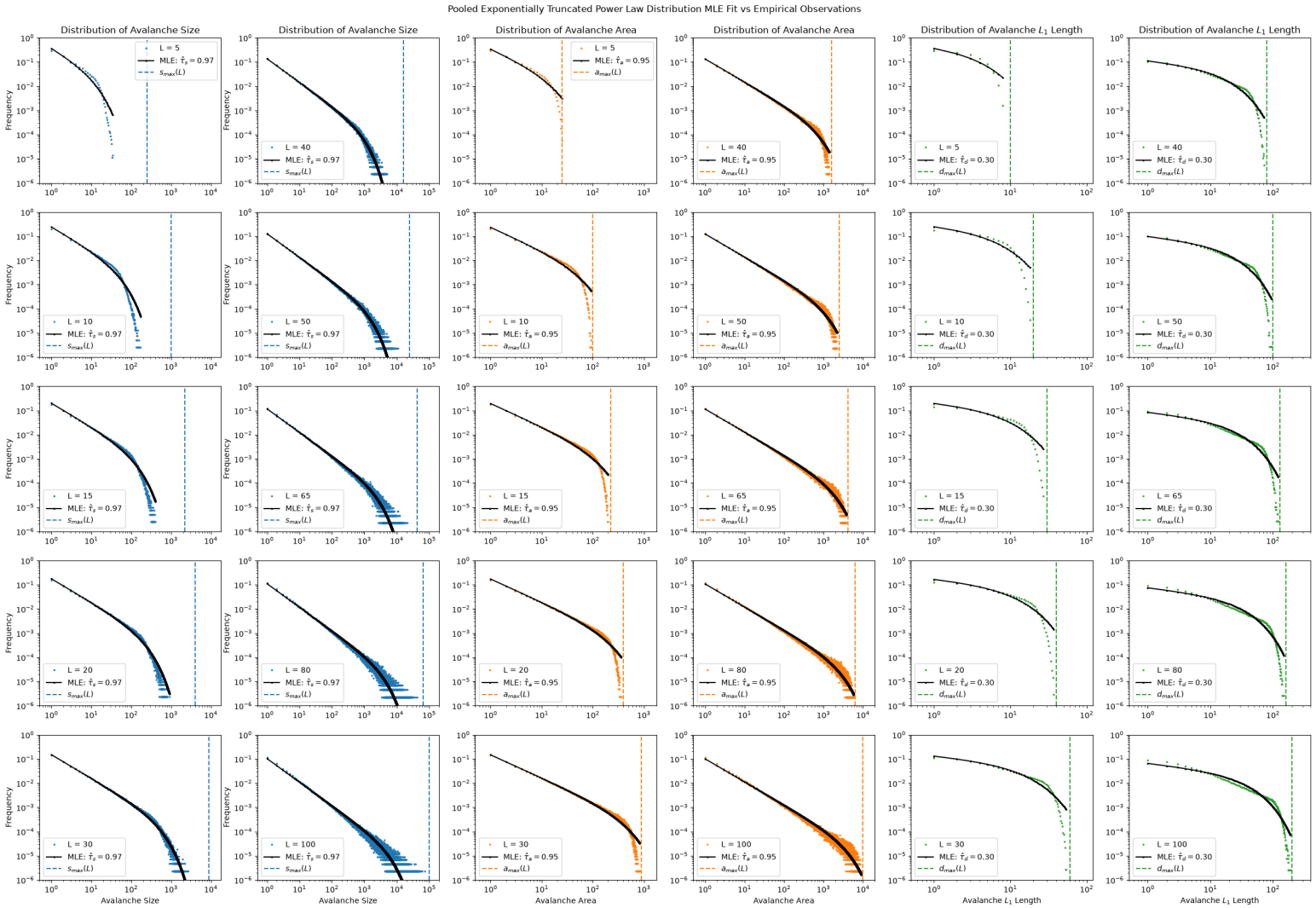


Figure A.4: Extended version of Figure 4.5 for all simulated lattice sizes.

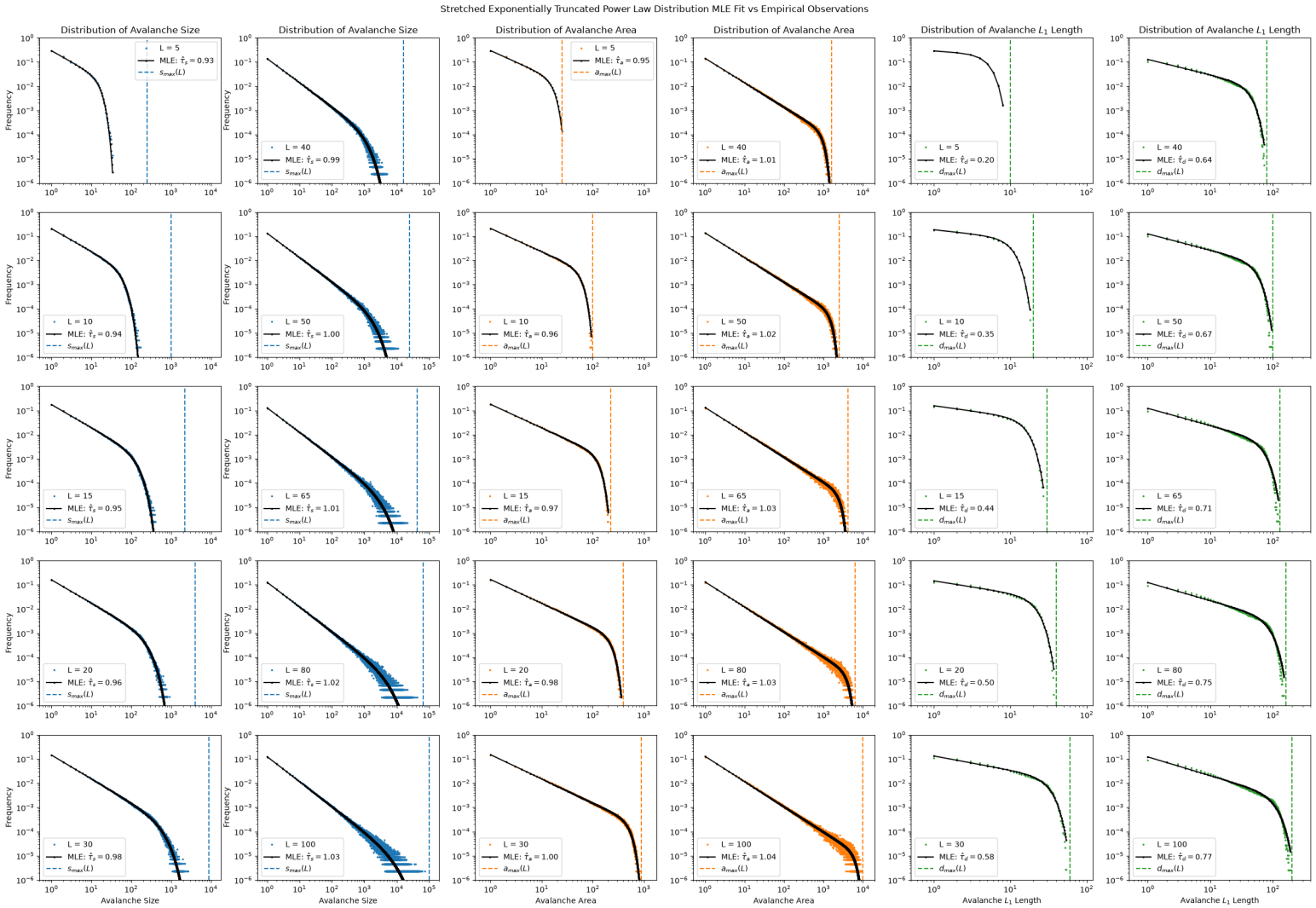


Figure A.5: Extended version of Figure 4.7 for all simulated lattice sizes.

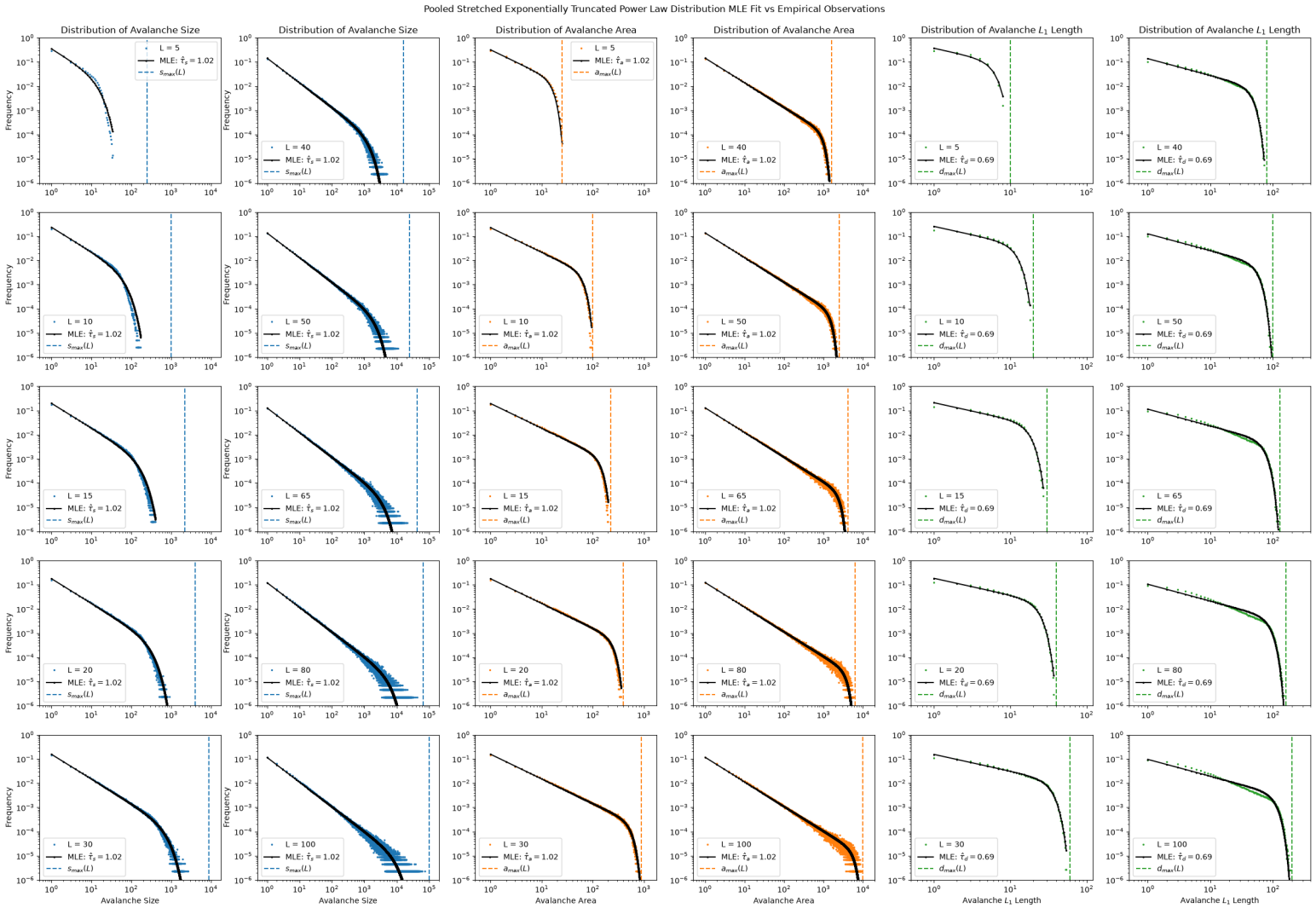


Figure A.6: Extended version of Figure 4.9 for all simulated lattice sizes.

Bibliography

[1] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality: An explanation of the 1/f noise. *Physical review letters*, 59(4):381, 1987.

[2] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality: An explanation of the 1/f noise. *Phys. Rev. Lett.*, 59:381–384, Jul 1987.

[3] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on scientific computing*, 16(5):1190–1208, 1995.

[4] D. Dhar. Self-organized critical state of sandpile automaton models. *Physical review letters*, 64(14):1613, 1990.

[5] A. A. Járai. Sandpile models. *Probability Surveys*, 15(none):243 – 306, 2018.

[6] S. K. Lam, A. Pitrou, and S. Seibert. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, pages 7:1–7:6, New York, NY, USA, 2015. ACM.